chuck-max cheatsheet

v0.3.0

Object

Form	Meaning
`[chuck~]`	1 channel in/out
`[chuck~ <N>]`	N channels in/out
`[chuck~ <N> <file>]`	N channels, run `<file>` at load
`[chuck~ <N> @ntap <M>]`	N channels + M tap outlets
`@verbose <0-2>`	object reporting: 0 quiet, 1 normal, 2 debug

Outlets, left to right: N audio, M tap, one reply (non-signal).

Core messages

Action	Max msg	Alias
Add shred from file, with optional args	`add <file> [arg1 arg2 ..]`	`+ <file> [args]`
Run file as shred (remembers last file)	`run <file>`	
Eval code as shred	`eval <code>`	
Remove shred	`remove <id>`	`- <id>`
Remove last shred	`remove last`	
Remove all shreds (keeps globals)	`remove all`	`removeall`
Replace shred	`replace <id> <file>`	`= <id> <file>`
Abort the shred executing inside the VM	`abort`	
List running shreds	`status`	
Clear VM (globals and types too)	`clear vm`	`reset`
Clear globals	`clear globals`	
Reset shred id counter	`reset id`	
Print VM time	`time`	

Utility messages

Action	Max msg
Set file attribute (does not run)	`file <path>`
Set editor path attribute	`editor <path>`
Prevent running shreds when DSP is off (default 1)	`run_needs_audio <0/1>`
Open file in external editor	`edit <path>`
List chugins	`chugins`
Get/set ChucK VM log level (0-10, process-wide)	`loglevel` / `loglevel <n>`
Get/set object verbosity (0-2)	`verbose` / `verbose <n>`
Get VM state	`vm`
Open ChucK docs in a browser	`docs`
Clear Max console	`clear console`
List all VM parameters	`param`
Get / set a VM parameter	`param <name>` / `param <name> <value>`
List shreds, or ready/blocked ones	`shreds` / `shreds ready` / `shreds blocked`
Shred detail	`shreds <id>`

Action	Max msg
Shred id queries	`shreds highest` / `last` / `next` / `count`
Get / set adaptive block size (0 off)	`adaptive` / `adaptive <size>`
Deliver a message to `HostIn` shreds	`msg <name> <args...>`

Tap outlets (`@ntap`)

Action	Max msg
Tap one global UGen from every outlet	`tap <ugen>`
Tap a UGen from one outlet (1-based)	`tap <outlet> <ugen>`
Tap a multichannel UGen across consecutive outlets	`tap <outlet> <ugen> <nchannels>`
Clear one outlet / all outlets	`tap <outlet>` / `tap`

In ChucK: `global Gain g => blackhole; 1 => g.buffered;`. Each sub-channel of a multichannel UGen needs its own `buffered` flag.

Globals and events

Action	ChucK	Max msg
Set global (untyped; must match declared type)	`global` variable	`<name> <value>`
List globals (console and reply outlet)	`global` variable	`globals`
Trigger event	`global Event`	`sig <name>`
Trigger event, all waiting shreds	`global Event`	`broadcast <name>`
Listen to event, one shot	`global Event`	`listen <name>` / `listen <name> 0`
Listen to event, forever	`global Event`	`listen <name> 1`
Stop listening	`global Event`	`unlisten <name>`

Typed get and set

`get` replies arrive at the reply outlet as `val <name> <value...>`.

Type	Get	Set
int	`get int <name>`	`set int <name> <v>`
float	`get float <name>`	`set float <name> <v>`
string	`get string <name>`	`set string <name> <v>`
int array	`get int[] <name>`	`set int[] <name> v1 v2 ..`
float array	`get float[] <name>`	`set float[] <name> v1 v2 ..`
int array, by index	`get int[i] <name> <i>`	`set int[i] <name> <i> <v>`
float array, by index	`get float[i] <name> <i>`	`set float[i] <name> <i> <v>`
int map, by key	`get int[k] <name> <key>`	`set int[k] <name> <key> <v>`
float map, by key	`get float[k] <name> <key>`	`set float[k] <name> <key> <v>`

Reply outlet (rightmost)

Message	Sent when
`val <name> <value...>`	a `get` completes
`event <name>`	a listened event fires
`shred add <id>` / `shred remove <id>`	a shred is sporked or removed
`global <name> <type>`	one per variable, in reply to `globals`
`msg <name> <args...>`	a shred calls `Host.msg()`

Route with `[route val event shred global msg]`. Replies are queued on the audio thread and flushed on the main thread; the queue drops when full.

Host classes (ChucK side)

Bound into every VM; shared with pd-chuck and numchuck. Examples: `examples/max/max_*.ck`.

Class	Call	Effect
`Host`	`Host.msg(name)`	reply outlet: `msg <name>`
`Host`	`Host.msg(name, float)` / `(name, string)` / `(name, float[])`	reply outlet: `msg <name> <args...>` (list max 63)
`Host`	`Host.sendBang(rcv)`	every `[receive rcv]`: `bang`
`Host`	`Host.sendFloat(rcv, f)` / `sendSymbol(rcv, s)`	`float f` / `symbol s`
`Host`	`Host.sendList(rcv, float[])`	`list ...` (max 32; floats only)
`HostIn`	`addAddress(name)`; `in => now`; `in.recv(msg)`	receives `msg <name> ...` from Max; extends `Event`
`HostMsg`	`name()`, `numArgs()`, `getInt(i)`, `getFloat(i)`, `getString(i)`	one received message
`HostBuffer`	`name(s)`, `ready()`, `frames()`, `channels()`, `sampleRate()`	a named `buffer~`; channels from 0
`HostBuffer`	`peek(frame[, chan])`, `poke(frame[, chan], value)`	read / write a sample
`HostBufPlay`	`name(s)`, `pos`, `rate`, `loop`, `interp`, `channel`	UGen playing one `buffer~` channel
`HostClock`	`ticks()`, `tempo()`, `running()`, `beat()`	Max global transport; extends `Event`
`HostClock`	`toNextBeat()`, `toNext(ticks)`	`dur` to next boundary; never zero
`HostClock`	`bar()`, `beatInBar()`, `unit()`	position at `now`; bar and beat from 1

`Host.msg` and `Host.send*` deliver on the main thread: control data, not a clock. `HostIn` delivers at the next audio block. `HostClock` broadcasts on start/stop, tempo or time signature change, and jumps.